TARGETED DRILL | ANALYTICS-DA-201

Calculations, LODs and Functions

Answer key with explanations

Questions	20 single-answer multiple-choice
Recommended time	35 minutes (about 1.75 minutes per question)
Focus	LOD grain, filter order, aggregate vs. row-level, functions
Product version	Tableau 2024.2
Format	Closed book | no Tableau | no notes

The GATE method

G	**Grain**	What rows must the number see? Row, customer, region, company?
A	**Aggregation**	Row-level, aggregate, LOD, or table calculation?
T	**Timing**	Before or after filters? Must hidden rows count?
E	**Example**	Test with 2-3 rows. Does the result make sense?

Each entry gives the correct option, the idea being tested, the reasoning, and the trap. For every miss, label the cause as grain, timing, or function.

Independent study material aligned to the published Analytics-DA-201 objectives. Not an official Salesforce or Tableau exam. Behavior reflects Tableau 2024.2.

Quick Answer Grid

Check your answers, then read every explanation - including the ones you got right.

Q	Q	Q	Q
1 C	6 D	11 B	16 B
2 B	7 C	12 A	17 D
3 A	8 D	13 B	18 A
4 C	9 C	14 B	19 D
5 C	10 A	15 D	20 A

The three checks

100% check: a share of 100% means the denominator grain equals the numerator grain.

Inflation check: a total that is too high suggests a one-to-many join or SUM of a repeated LOD.

Filter check: if a filter changes a result that should be fixed, the calculation is running after the filter.

Detailed Explanations

20 questions | Read the reasoning, not just the letter

Q1 ANSWER C

FIXED LOD and filters

C. Total sales for all regions in the data source.

Why: A `FIXED` LOD with no dimensions computes at the whole-data-source level. Ordinary dimension filters are applied after the FIXED value is computed, so the denominator is the company total, not the East total. The numerator is East-only because it is a normal view-level aggregate.

Exam trap: If `Region = East` were a **context** filter, the FIXED LOD would respect it and the denominator would become East's total. Context, data source, and extract filters can change a FIXED result; ordinary dimension filters do not.

Q2 ANSWER B

INCLUDE vs. EXCLUDE grain

B. `SUM([Sales]) / SUM({EXCLUDE [Category] : SUM([Sales])})`

Why: The view is at Region + Category. To get the parent (Region) total, remove Category from the calculation grain with `EXCLUDE [Category]`. The denominator becomes the region total, so each category is a share of its own region.

Exam trap: `INCLUDE [Category]` keeps the calculation at the row grain (Region + Category), so the denominator equals the category itself and the result is 100%. `FIXED [Category]` also gives 100%. `FIXED` with no dimensions gives the grand total across all regions.

Q3 ANSWER A

INCLUDE LOD / average of totals

A. `AVG({INCLUDE [Customer ID] : SUM([Sales])})`

Why: `INCLUDE [Customer ID]` computes each customer's total sales at Region + Customer granularity, even though Customer is not in the view. The outer `AVG` then averages those customer totals within each region. This is the correct 'average of totals' pattern.

Exam trap: `AVG([Sales])` averages individual order rows. `SUM/COUNT` divides by order rows, not distinct customers. If each customer has several orders, both give the wrong answer.

Q4 ANSWER C

Nested LODs

C. The highest city-level sales within each region.

Why: Nested LODs evaluate inside-out. The inner LOD computes sales at Region + City. The outer LOD then takes the maximum of those city values within each region. The result is the top city's sales for that region, repeated on rows in the region.

Exam trap: The outer aggregation is `MAX`, not `SUM`, so the result is not regional total sales. It is scoped by Region, so it is not a global average. Read nested LODs from the innermost expression outward.

Q5 ANSWER C

Aggregating a repeated FIXED LOD

C. 2,700

Why: The FIXED LOD computes 900 once per region, but Tableau stamps that value on every underlying row that belongs to the region. There are three such rows, and the outer `SUM` adds all three copies: 900 + 900 + 900 = **2,700**. This is the classic repeated-LOD inflation trap.

Exam trap: To show 900, aggregate the LOD with `AVG`, `MIN`, `MAX`, or `ATTR` instead of `SUM` (each returns 900 for the region), or use `SUM([Sales])` directly at the region level.

Q6 ANSWER D

FIXED LOD vs. measure filters

D. They remain the full region totals, because FIXED is computed before measure filters.

Why: FIXED LODs are computed early in the order of operations, before dimension and measure filters. The measure filter can hide marks, but it does not recompute the already-computed FIXED values. Visible regions keep their full region totals.

Exam trap: A measure filter changes which marks are displayed, not what a FIXED LOD calculated. Only context, data source, and extract filters run early enough to change the FIXED universe.

Q7 ANSWER C

Context filters and FIXED LODs

C. It is recalculated using only Office-category rows.

Why: Context filters run before FIXED LODs. By promoting the Category filter to context, the analyst changes the rows that the FIXED calculation sees. Each region's value is now the region's Office-category total, not its all-category total.

Exam trap: FIXED ignores ordinary dimension and measure filters, but it does **not** ignore context, data source, or extract filters. Context is still a filter; it simply runs earlier.

Q8 ANSWER D

FIXED LODs and NULL members

D. NULL is treated as its own partition and gets its own LOD value.

Why: A FIXED LOD groups by the values of the specified dimension, and NULL is treated as a distinct member. The LOD creates a separate partition for NULL and computes its own sum, just as it would for East or West.

Exam trap: If you want NULLs handled differently, replace them before the LOD with `IFNULL` or exclude them with a filter. By default, they are neither dropped nor merged into other regions.

Q9 ANSWER C

Aggregate vs. row-level ratios

C. `SUM([Sales]) / SUM([Quantity])`

Why: The weighted ratio is total sales divided by total quantity: (100 + 300 + 200) / (2 + 4 + 1) = 600 / 7 = **85.71**. Aggregating the numerator and denominator separately preserves the business weighting.

Exam trap: `AVG([Sales] / [Quantity])` is the unweighted average of the row-level ratios: (50 + 75 + 200) / 3 = 108.33. That answers 'average of ratios', not 'revenue per unit across all orders'. The exam wording 'overall / across all / weighted' points to `SUM/SUM`.

Q10 ANSWER A

COUNT vs. COUNTD

A. `SUM([Sales]) / COUNTD([Order ID])`

Why: Average sales per distinct order = total sales divided by the number of distinct orders: 600 / 3 = **200**. `COUNTD([Order ID])` counts the three unique order IDs.

Exam trap: `COUNT([Sales])` counts the five order lines, giving 120. `AVG([Sales])` also averages the five lines. `COUNTD([Customer ID])` gives 300, which is sales per customer, not per order.

Q11 ANSWER B

ATTR

B. An asterisk (*).

Why: `ATTR()` returns the value when all rows in the partition share one value. When multiple distinct values exist, it returns `*` to signal that the value is not unique at that level of detail.

Exam trap: ATTR does not pick a first value, concatenate values, or return NULL. It is a quick way to detect whether a dimension is unique at the current aggregation level.

Q12 ANSWER A

DATEDIFF boundaries

A. 2

Why: `DATEDIFF` counts the number of date-part **boundaries crossed**, not the number of complete elapsed months. From January 31 to March 1, the February and March month boundaries are crossed, so the result is **2**.

Exam trap: Many candidates calculate one full elapsed month and answer 1. Tableau's DATEDIFF is boundary-based, which is a favorite exam trap.

Q13 ANSWER B

Date calculation: end of previous month

B. `DATEADD('day', -1, DATETRUNC('month', TODAY()))`

Why: `DATETRUNC('month', TODAY())` gives the first day of the current month. Subtracting one day returns the day before that: the last day of the previous month.

Exam trap: Truncating yesterday to the month returns the first of yesterday's month, which is usually the first of the current month. `DATEADD('month', -1, TODAY())` returns the same day one month earlier, not the month end. `DATETRUNC('month', TODAY())` is the first of the current month, not the last day of the previous month.

Q14 ANSWER B

String extraction

B. `MID([ID], 10, 4)`

Why: `MID(string, start, length)` starts at position 1. In `INV-2025-0042-X`, position 10 is the first `0` of `0042`, and a length of 4 returns `0042`.

Exam trap: `LEFT(...,4)` returns `INV-`. The last six characters include `0042-X`, not just `0042`. `FIND` returns the position of the first hyphen (4), not the substring.

Q15 ANSWER D

REPLACE behavior

D. bonono

Why: `REPLACE` substitutes **every** occurrence of the search string. All three `a` characters in `banana` become `o`, giving `bonono`. There is no 'first occurrence only' flag.

Exam trap: `bonana` and `banano` assume only one occurrence is replaced. Tableau replaces all occurrences.

Q16 ANSWER B

IIF and NULL logic

B. `Unknown`

Why: `IIF` has an optional fourth argument for the unknown/NULL result. When the comparison with a NULL date evaluates to unknown, `IIF` returns the fourth argument, `'Unknown'`.

Exam trap: A plain `IF` with a NULL condition falls through to `ELSE` (or returns NULL if there is no ELSE). `IIF` handles unknown separately through its fourth argument, which is exactly what this question tests.

Q17 ANSWER D

ZN vs. IFNULL

D. `IFNULL([Country], 'Unknown')`

Why: `IFNULL` works with any data type and any replacement value. It returns the original value when it is not null, and the replacement when it is null.

Exam trap: `ZN` is numeric only, so it fails on text. `ISNULL` returns a boolean, not a replacement value. Comparing a field to NULL with `=` does not work in Tableau; use `ISNULL()`.

Q18 ANSWER A

INT vs. FLOOR

A. `FLOOR(-7.2)`

Why: `FLOOR` rounds down toward negative infinity, so `FLOOR(-7.2)` returns **-8**. `INT` truncates toward zero and returns -7. `ROUND(-7.2, 0)` returns -7. `CEILING(-7.2)` returns -7.

Exam trap: Do not treat INT and FLOOR as synonyms for negative numbers. INT truncates toward zero; FLOOR always goes down.

Q19 ANSWER D

Table calculation addressing and partitioning

D. Address by Month and partition by Year.

Why: Addressing sets the direction the running total moves; partitioning sets the groups that reset it. To accumulate across months but restart for each year, address the calculation by Month and partition by Year.

Exam trap: Addressing by Year and partitioning by Month does the opposite: it resets every month and accumulates across years. A plain SUM is not cumulative, and a filter does not create a running total.

Q20 ANSWER A

Spatial calculation: BUFFER

A. `BUFFER(MAKEPOINT([Lat], [Lon]), 10, 'mi')`

Why: `BUFFER(point, distance, units)` creates a polygon centered on a point with the specified radius. It is the correct spatial calculation for a delivery-radius ring.

Exam trap: `MAKEPOINT` creates only a point. `MAKELINE` connects two points with a line. `DISTANCE` returns a scalar length between two points, not a polygon.